

Apunte teórico: Desarrollo de Dashboards Interactivos con Streamlit

Llegamos a la etapa final del proyecto. Ya sabemos extraer datos de una base de datos MySQL, sabemos limpiarlos con Pandas y generar métricas de negocio. Sin embargo, el cliente final (un gerente, un usuario, un inversor) no va a ejecutar nuestro código fuente ni va a leer la consola. Necesita una interfaz gráfica, amigable e interactiva.

Aquí es donde entra **Streamlit**. Es una librería de código abierto que nos permite transformar scripts de Python en aplicaciones web interactivas en cuestión de minutos, sin necesidad de tener conocimientos previos en diseño web (HTML, CSS o JavaScript).

1. El Paradigma de Streamlit: ¿Cómo funciona?

A diferencia de un Jupyter Notebook que se ejecuta celda por celda a voluntad del programador, Streamlit tiene un **flujo de ejecución lineal y reactivo**:

1. El código se lee siempre de arriba hacia abajo, de corrido.
2. Cada vez que el usuario interactúa con la pantalla (por ejemplo, cambia un filtro, mueve un deslizador o hace clic en un botón), Streamlit **vuelve a ejecutar todo el script desde la línea 1**.
3. Esto asegura que los datos y los gráficos en pantalla estén siempre actualizados con la última interacción del usuario.

2. Diccionario de Componentes Clave

Para construir la interfaz de nuestro TPI, no necesitamos memorizar cientos de comandos. Streamlit agrupa sus funciones de manera muy intuitiva. A continuación, las herramientas fundamentales para armar el Dashboard:

Categoría	Comando en Python	Función en la Interfaz Web
Estructura y Texto	st.title("Texto")	Crea el título principal de la aplicación.
	st.markdown("Texto")	Permite escribir texto con formato (negritas, subtítulos, listas) usando sintaxis Markdown.
	st.divider()	Dibuja una línea horizontal separadora para ordenar visualmente el contenido.
Disposición (Layout)	st.sidebar	Habilita un panel lateral izquierdo. Ideal para colocar todos los filtros de la aplicación.
	col1, col2 = st.columns(2)	Divide el ancho de la pantalla en columnas para ubicar gráficos o KPIs uno al lado del otro.
Métricas (KPIs)	st.metric(label, value)	Crea una tarjeta de indicador clave (ej. "Ventas Totales" en tamaño grande).

Tablas de Datos	<code>st.dataframe(df)</code>	Renderiza un DataFrame de Pandas en formato de tabla interactiva (permite ordenar y hacer scroll).
------------------------	-------------------------------	--

3. Filtros y Controladores (Widgets)

El verdadero valor de un Dashboard es la interactividad. Los widgets son los elementos que le permiten al usuario filtrar la información. Al asignarle un widget a una variable, podemos usar esa variable para filtrar nuestro DataFrame de Pandas.

- **Menú Desplegable (Selectbox):** Ideal para elegir una opción entre varias (ej. seleccionar un barrio o una provincia).

```
barrio = st.selectbox("Elija un barrio:", options=["Centro", "Sur", "Norte"])
```

- **Selección Múltiple (Multiselect):** Permite elegir varias opciones al mismo tiempo. Devuelve una lista de Python.

```
categorias = st.multiselect("Filtre por categorías:", options=["Tecnología", "Hogar", "Bazar"])
```

- **Deslizador Numérico (Slider):** Excelente para filtrar por rangos de fechas o montos de dinero.

```
rango_precio = st.slider("Seleccione el rango de precio:", min_value=0, max_value=10000)
```

4. Visualización de Gráficos Integrados

Streamlit se lleva excelente con las librerías de visualización que ya conocemos.

- **Gráficos Nativos Rápidos:** Si tienen un DataFrame ya agrupado por tiempo, pueden usar `st.line_chart(df)` o `st.bar_chart(df)` para dibujar un gráfico interactivo en un solo paso.
- **Integración con Matplotlib / Seaborn:** Si diseñaron un mapa de calor o un gráfico muy específico con Seaborn, pueden pasarlo a Streamlit utilizando la función `st.pyplot(fig)`.

5. El Motor de Rendimiento: Caché (@st.cache_data)

Este es **el concepto más importante** para que su Trabajo Práctico Integrador funcione correctamente y no colapse la base de datos.

Como dijimos, Streamlit vuelve a ejecutar todo el código cada vez que tocamos un filtro. Si en nuestro código tenemos la función `pd.read_sql_query()` conectada a MySQL, iStreamlit le haría una consulta nueva a la base de datos cada vez que el usuario haga un clic! Esto es ineficiente y muy lento.

Para evitarlo, utilizamos el decorador `@st.cache_data` justo arriba de la función que carga nuestros datos. Esto le dice a Streamlit: *"Ejecutá esta consulta SQL solo la primera vez que se abra la app. Guardá los resultados en la memoria temporal (Caché). Las próximas veces que el usuario toque un filtro, usá los datos guardados en lugar de volver a molestar al servidor MySQL"*.

```
@st.cache_data
```

```
def cargar_datos_desde_mysql():
```

```
engine = create_engine("mysql+pymysql://usuario:clave@localhost:3306/mi_base")

df = pd.read_sql_query("SELECT * FROM ventas", con=engine)

return df
```

Esta línea solo consultará a MySQL la primera vez

df_ventas = cargar_datos_desde_mysql()

Recomendaciones Arquitectónicas para el TPI

1. **Mantengan el archivo limpio:** Su archivo app.py debe enfocarse en la interfaz. Traten de que la limpieza pesada de datos y la estructuración de la base se hagan previamente en MySQL o en un script de preparación.
2. **No suban claves a la nube:** Durante la etapa de desarrollo local, escribir root o contraseñas vacías está bien. Cuando vayan a entornos profesionales, nunca deben dejar estas credenciales escritas en el código.
3. **Cuenten una historia:** No amontonen gráficos sin sentido. Arriba los KPIs globales, en el medio los gráficos comparativos (tendencias, barras) y abajo las tablas de detalle para quien necesite auditar los registros.

6. Despliegue en la Nube (Streamlit Community Cloud)

¡El código en sus computadoras funciona perfecto! Pero un Dashboard no sirve de mucho si solo vive en sus discos duros. El último paso como desarrolladores es hacer el **Despliegue (Deploy)** de su aplicación web a la nube para que cualquier persona en el mundo pueda acceder a ella a través de un link.

Para esto, utilizaremos **Streamlit Community Cloud**, un servicio gratuito que aloja nuestras aplicaciones de forma automática.

Paso 1: El archivo de dependencias (requirements.txt)

Cuando subimos nuestro código a la nube, la computadora de Streamlit está "vacía". Necesitamos decirle qué librerías externas debe instalar para que nuestro código no falle.

1. En la misma carpeta donde está su archivo app.py, creen un archivo de texto plano llamado exactamente **requirements.txt**.
2. Escriban dentro el nombre de las librerías que usaron, una por línea:

streamlit

pandas

sqlalchemy

pymysql

(Agreguen cualquier otra librería extra que hayan importado, como seaborn o matplotlib)

Paso 2: Subir el proyecto a GitHub

Streamlit Cloud lee el código directamente desde sus repositorios.

1. Creen un repositorio público en su cuenta de GitHub (ej. tpi-dashboard-logistica).
2. Suban sus archivos principales: app.py y requirements.txt. *(Asegúrense de no subir entornos virtuales ni archivos innecesarios).*

Paso 3: Conectar y Desplegar

1. Ingresen a share.streamlit.io e inicien sesión vinculando su cuenta de GitHub.
2. Hagan clic en el botón **"New app"**.
3. Seleccionen el repositorio que acaban de crear, la rama (suele ser main o master) y el nombre de su archivo principal (app.py).
4. Hagan clic en **Deploy!**
5. Esperen un par de minutos mientras el servidor lee su requirements.txt e instala todo. Al finalizar, ¡tendrán una URL pública lista para compartir en su presentación!

¡CUIDADO! El problema del localhost en la Nube

Si su código tiene esta línea: engine =

create_engine("mysql+pymysql://root:@localhost:3306/mi_base") **¡La aplicación va a fallar en la nube!**

¿Por qué? Como todos saben, en el entorno de la nube, la palabra localhost se refiere al propio servidor de Streamlit, no a sus computadoras. Streamlit Cloud no tiene instalado XAMPP ni tiene su base de datos local.

¿Cómo lo solucionamos para el TPI?

Tienen dos opciones de arquitectura profesional:

- **Opción A (La ruta Cloud Native):** Migrar su base de datos MySQL local a un servicio de bases de datos gratuito en la nube (como *Aiven*, *Clever Cloud* o *Supabase*). Esos servicios les darán un *Host* público, un usuario y una contraseña reales para reemplazar el localhost en su código.
- **Opción B (El plan de contingencia):** Si no llegan a configurar una base de datos en la nube para la fecha de la entrega, pueden exportar sus datos limpios de MySQL a un archivo .csv, subir ese archivo a GitHub junto con su código, y modificar la función de carga en su app.py para que lea el CSV (pd.read_csv()) en lugar de conectarse por SQL. Así se aseguran de que la interfaz gráfica interactiva funcione perfecto frente al tribunal evaluador.